

AI Assisted Coding Assignment- 13.5

RUDROJU RUPA SRI

2303A51918

BATCH-30

Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions

Lab Objectives:

- Identify code smells and inefficiencies in legacy Python scripts.
- Use AI-assisted coding tools to refactor for readability, maintainability, and performance.
- Apply modern Python best practices while ensuring output correctness.

Task Description #1 (Refactoring – Removing Global Variables)

- Task: Use AI to eliminate unnecessary global variables from the code.

- Instructions:

- o Identify global variables used across functions.
- o Refactor the code to pass values using function parameters.
- o Improve modularity and testability.

- Sample Legacy Code:

```
rate = 0.1
```

```
def calculate_interest(amount):  
    return amount * rate  
print(calculate_interest(1000))
```

- Expected Output:

- o Refactored version passing rate as a parameter or using a configuration structure.

```
docs | C:\Users\HP\Desktop\AI\ai_ass_9.5.py
1 #Task Description #1 (Refactoring - Removing Global Variables)
2 #Sample Legacy Code:
3 #rate = 0.1
4 #def calculate_interest(amount):
5 #return amount * rate
6 #print(calculate_interest(1000))
7 #Prompt: Refactor the code to pass values using function parameters.
8 def calculate_interest(amount, rate):
9     """
10     Calculate the interest based on the given amount and rate.
11
12     Parameters:
13     amount (float): The principal amount for which interest is to be calculated.
14     rate (float): The interest rate to be applied.
15
16     Returns:
17     float: The calculated interest.
18
19     Example:
20     >>> calculate_interest(1000, 0.1)
21     100.0
22     """
23     return amount * rate
24 # Example usage
25 print(calculate_interest(1000, 0.1)) # Expected output: 100.0
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

● PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "c:\Users\HP\Desktop\AI\docs\AI_ASS-13.5.py"

100.0

○ PS C:\Users\HP\Desktop\AI> |

powerShell Python

Describe what to build next

Ln 25, Col 63 Spaces: 4 UTF-8 CRLF Python Python 3.13 (64-bit) Go Live

Task Description #2 : (Refactoring Deeply Nested Conditionals)

- Task: Use AI to refactor deeply nested if-elif-else logic into a cleaner structure.

- Focus Areas:

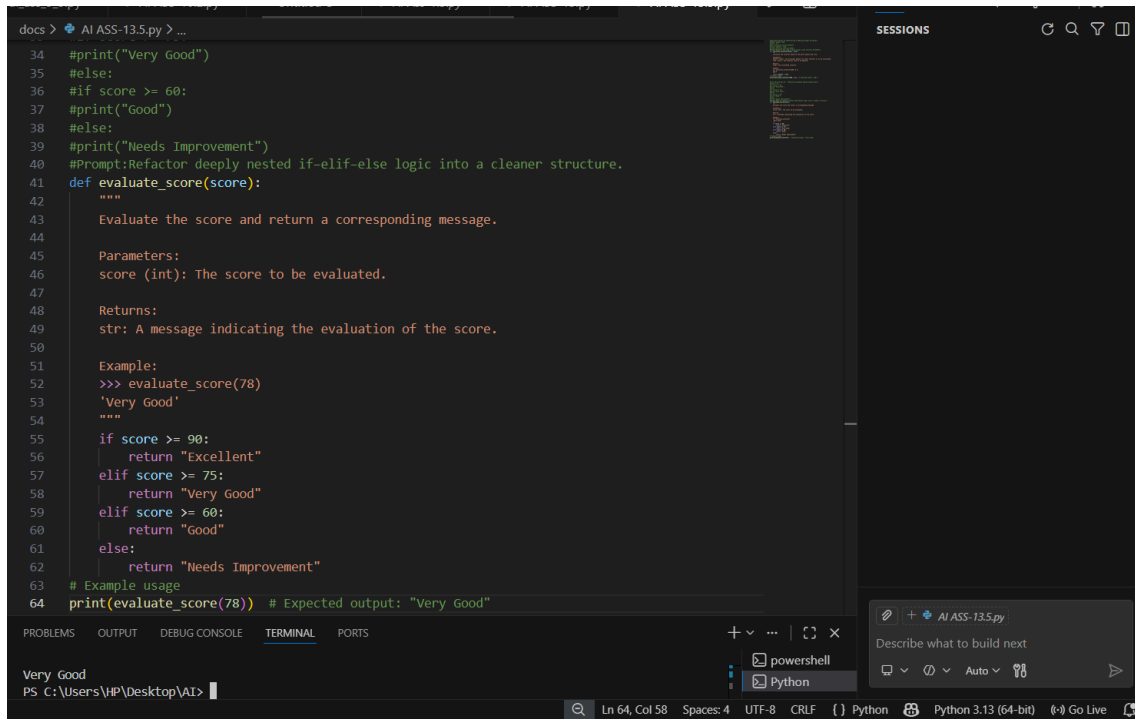
- o Readability
- o Logical simplification
- o Maintainability

Legacy Code:

```
score = 78
if score >= 90:
    print("Excellent")
else:
    if score >= 75:
        print("Very Good")
    else:
        if score >= 60:
            print("Good")
        else:
            print("Needs Improvement")
```

Expected Outcome:

- o Flattened logic using guard clauses or a mapping-based approach.



```
docs > AI ASS-13.5.py > ...
34 #print("Very Good")
35 #else:
36 #if score >= 60:
37 #print("Good")
38 #else:
39 #print("Needs Improvement")
40 #Prompt: Refactor deeply nested if-elif-else logic into a cleaner structure.
41 def evaluate_score(score):
42     """
43     Evaluate the score and return a corresponding message.
44
45     Parameters:
46     score (int): The score to be evaluated.
47
48     Returns:
49     str: A message indicating the evaluation of the score.
50
51     Example:
52     >>> evaluate_score(78)
53     'Very Good'
54     """
55     if score >= 90:
56         return "Excellent"
57     elif score >= 75:
58         return "Very Good"
59     elif score >= 60:
60         return "Good"
61     else:
62         return "Needs Improvement"
63 # Example usage
64 print(evaluate_score(78)) # Expected output: "Very Good"
```

Very Good
PS C:\Users\HP\Desktop\AI>

Task 3 (Refactoring Repeated File Handling Code)

- Task: Use AI to refactor repeated file open/read/close logic.

- Focus Areas:

- o DRY principle
- o Context managers
- o Function reuse

Legacy Code:

```
f = open("data1.txt")
```

```
print(f.read())
```

```
f.close()
```

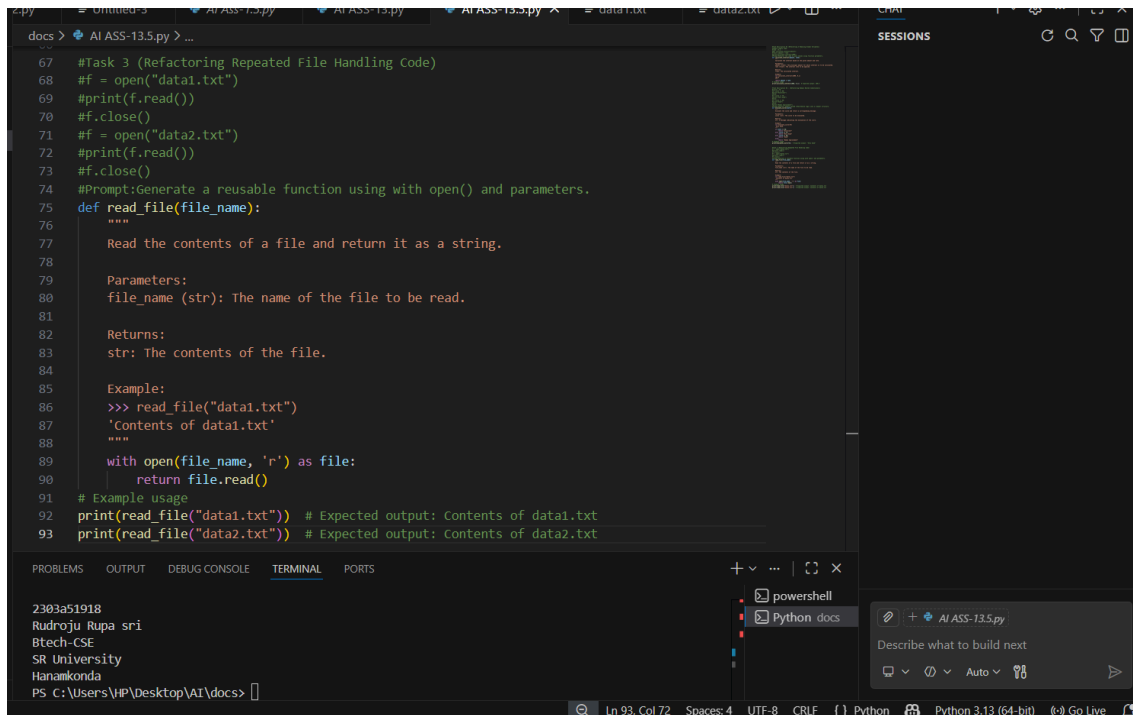
```
f = open("data2.txt")
```

```
print(f.read())
```

```
f.close()
```

Expected Outcome:

- o Reusable function using with open() and parameters.



```
67 #Task 3 (Refactoring Repeated File Handling Code)
68 #f = open("data1.txt")
69 #print(f.read())
70 #f.close()
71 #f = open("data2.txt")
72 #print(f.read())
73 #f.close()
74 #Prompt:Generate a reusable function using with open() and parameters.
75 def read_file(file_name):
76     """
77     Read the contents of a file and return it as a string.
78
79     Parameters:
80     file_name (str): The name of the file to be read.
81
82     Returns:
83     str: The contents of the file.
84
85     Example:
86     >>> read_file("data1.txt")
87     'Contents of data1.txt'
88     """
89     with open(file_name, 'r') as file:
90         return file.read()
91 # Example usage
92 print(read_file("data1.txt")) # Expected output: Contents of data1.txt
93 print(read_file("data2.txt")) # Expected output: Contents of data2.txt
```

2303a51918
Rudroju Rupa sri
Btech-CSE
SR University
Hanamkonda
PS C:\Users\HP\Desktop\AI\docs>

Task 4 (Optimizing Search Logic)

- Task: Refactor inefficient linear searches using appropriate data structures.

- Focus Areas:

- o Time complexity
- o Data structure choice

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]
```

```
name = input("Enter username: ")
```

```
found = False
```

```
for u in users:
```

```
if u == name:
```

```
found = True
```

```
print("Access Granted" if found else "Access Denied")
```

Expected Outcome:

- o Use of sets or dictionaries with complexity justification.

The screenshot shows a code editor with a dark theme. The main editor area displays Python code for Task 4, which is a login script. The code is as follows:

```
95
96 #Task 4 (Optimizing Search Logic)
97 #users = ["admin", "guest", "editor", "viewer"]
98 #name = input("Enter username: ")
99 #found = False
100 #for u in users:
101 #if u == name:
102 #found = True
103 #print("Access Granted" if found else "Access Denied")
104 #Prompt:Refactor inefficient linear searches using appropriate data structures
105 users = {"admin": "Administrator", "guest": "Guest User", "editor": "Editor"}
106 name = input("Enter username: ")
107 if name in users:
108     print("Access Granted")
109 else:
110     print("Access Denied")
```

Below the code editor is a terminal window. It shows the command prompt with the following input and output:

```
Enter username: rupa
Access Denied
PS C:\Users\HP\Desktop\AI\docs>
```

The terminal window also shows a list of open files: "powershell" and "Python docs". The status bar at the bottom indicates the current line and column (Ln 110, Col 27), the number of spaces (Spaces: 4), the encoding (UTF-8), the line ending (CRLF), the language (Python), and the Python version (Python 3.13 (64-bit)).

Task 5 (Refactoring Procedural Code into OOP Design)

- Task: Use AI to refactor procedural code into a class-based design.

- Focus Areas:

- o Object-Oriented principles
- o Encapsulation

Legacy Code:

```
salary = 50000
```

```
tax = salary * 0.2
```

```
net = salary - tax
```

```
print(net)
```

Expected Outcome:

- o A class like EmployeeSalaryCalculator with methods and attributes.

```

112
113 #Task 5 (Refactoring Procedural Code into OOP Design)
114 class EmployeeSalaryCalculator:
115     """
116     A class to calculate the net salary of an employee after tax
117     """
118     def __init__(self, salary, tax_rate=0.2):
119         self.salary = salary
120         self.tax_rate = tax_rate
121     def calculate_tax(self):
122         return self.salary * self.tax_rate
123     def calculate_net_salary(self):
124         return self.salary - self.calculate_tax()
125 # Example usage
126 employee = EmployeeSalaryCalculator(50000)
127 print(employee.calculate_net_salary())

```

40000.0

PS C:\Users\HP\Desktop\AI\docs>

Ln 118, Col 49 Spaces: 4 UTF-8 CRLF Python Python 3.13 (64-bit) Go Live

Task 6 (Refactoring for Performance Optimization)

- Task: Use AI to refactor a performance-heavy loop handling large data.

- Focus Areas:

- o Algorithmic optimization
- o Use of built-in functions

Legacy Code:

total = 0

for i in range(1, 1000000):

if i % 2 == 0:

total += i

print(total)

Expected Outcome:

- o Optimized logic using mathematical formulas or comprehensions, with time comparison.

```

128
129 #Task 6 (Refactoring for Performance Optimization)
130 #total = 0
131 #for i in range(1, 1000000):
132 #if i % 2 == 0:
133 #total += i
134 #print(total)
135 #Prompt:Generate Optimized logic code using mathematical formulas
136 total = sum(i for i in range(1, 1000000) if i % 2 == 0)
137 print(total) # Expected output: 249999500000

```

249999500000

PS C:\Users\HP\Desktop\AI\docs>

Ln 136, Col 49 Spaces: 4 UTF-8 CRLF Python Python 3.13 (64-bit) Go Live

Task 7 (Removing Hidden Side Effects)

- Task: Refactor code that modifies shared mutable state.

- Focus Areas:

- o Functional-style refactoring

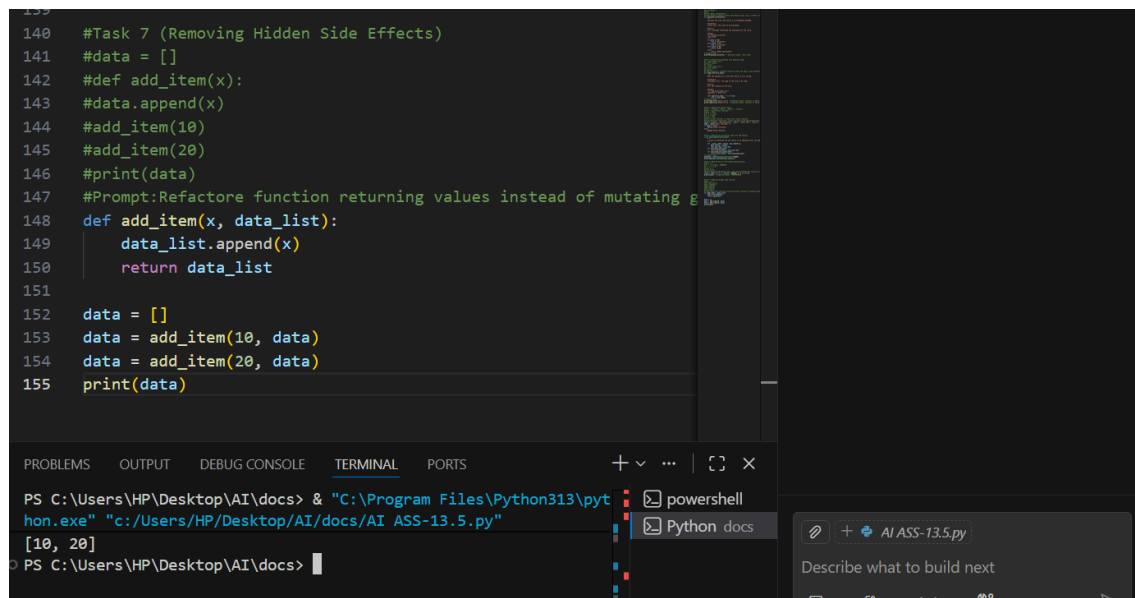
- o Predictability

Legacy Code:

```
data = []  
  
def add_item(x):  
    data.append(x)  
    add_item(10)  
    add_item(20)  
    print(data)
```

Expected Outcome:

- o Refactored function returning values instead of mutating globals.



```
140 #Task 7 (Removing Hidden Side Effects)  
141 #data = []  
142 #def add_item(x):  
143 #    data.append(x)  
144 #    add_item(10)  
145 #    add_item(20)  
146 #    print(data)  
147 #Prompt: Refactore function returning values instead of mutating g  
148 def add_item(x, data_list):  
149     data_list.append(x)  
150     return data_list  
151  
152 data = []  
153 data = add_item(10, data)  
154 data = add_item(20, data)  
155 print(data)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\HP\Desktop\AI\docs> & "C:\Program Files\Python313\python.exe" "c:/Users/HP/Desktop/AI/docs/AI_ASS-13.5.py"
[10, 20]
PS C:\Users\HP\Desktop\AI\docs>

Task 8 (Refactoring Complex Input Validation Logic)

- Task: Use AI to simplify and modularize complex validation rules.

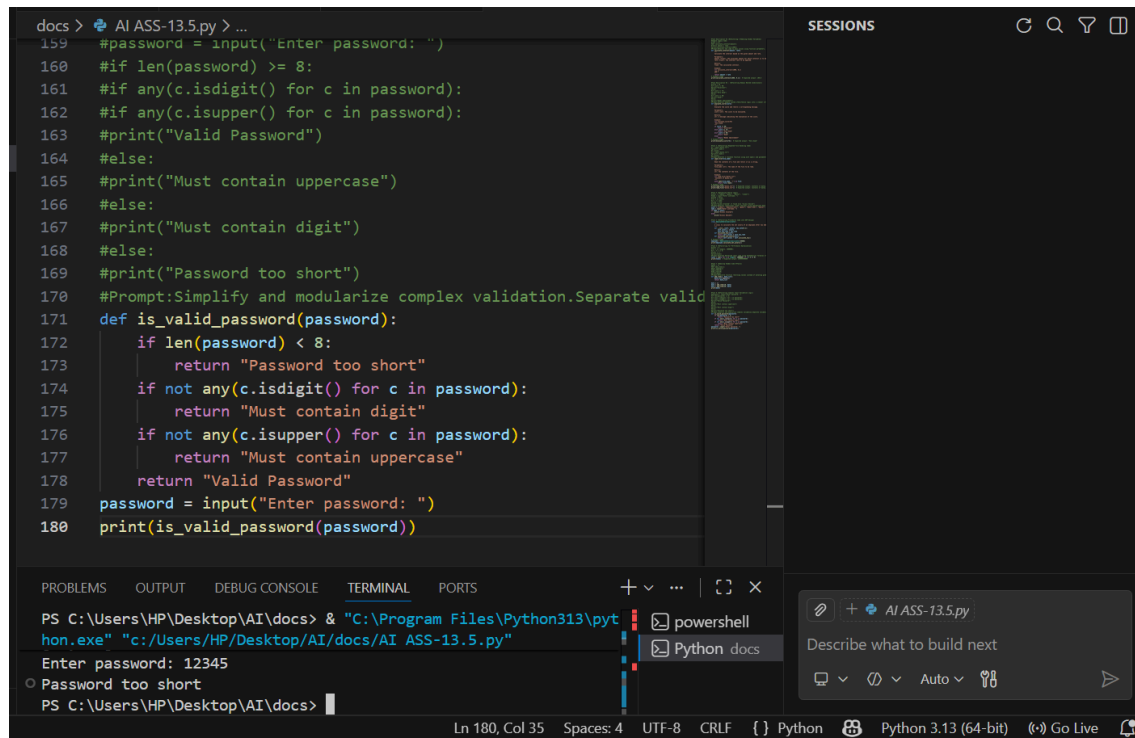
- Focus Areas:

- o Readability

- o Testability

Legacy Code:

```
password = input("Enter password: ")
if len(password) >= 8:
    if any(c.isdigit() for c in password):
        if any(c.isupper() for c in password):
            print("Valid Password")
        else:
            print("Must contain uppercase")
    else:
        print("Must contain digit")
    else:
        print("Password too short")
Expected Outcome:
o Separate validation functions with clear responsibility
```



The screenshot shows a Visual Studio Code editor with a Python file named `AI ASS-13.5.py`. The code implements a password validation function `is_valid_password` and a main loop that prompts the user for a password and checks it using the function. The code is as follows:

```
159 #password = input("Enter password: ")
160 #if len(password) >= 8:
161 #if any(c.isdigit() for c in password):
162 #if any(c.isupper() for c in password):
163 #print("Valid Password")
164 #else:
165 #print("Must contain uppercase")
166 #else:
167 #print("Must contain digit")
168 #else:
169 #print("Password too short")
170 #Prompt:Simplify and modularize complex validation.Separate valid
171 def is_valid_password(password):
172     if len(password) < 8:
173         return "Password too short"
174     if not any(c.isdigit() for c in password):
175         return "Must contain digit"
176     if not any(c.isupper() for c in password):
177         return "Must contain uppercase"
178     return "Valid Password"
179 password = input("Enter password: ")
180 print(is_valid_password(password))
```

The terminal output shows the execution of the script:

```
PS C:\Users\HP\Desktop\AI\docs> & "C:\Program Files\Python313\python.exe" "c:/Users/HP/Desktop/AI/docs/AI ASS-13.5.py"
Enter password: 12345
Password too short
PS C:\Users\HP\Desktop\AI\docs>
```

The interface also includes a sidebar with 'SESSIONS', a 'PROBLEMS' panel, and a 'TERMINAL' panel. The status bar at the bottom indicates the current line and column (Ln 180, Col 35), the number of spaces (4), the encoding (UTF-8), the line ending (CRLF), and the active language (Python).